



死锁

定义

必要条件

处理方法

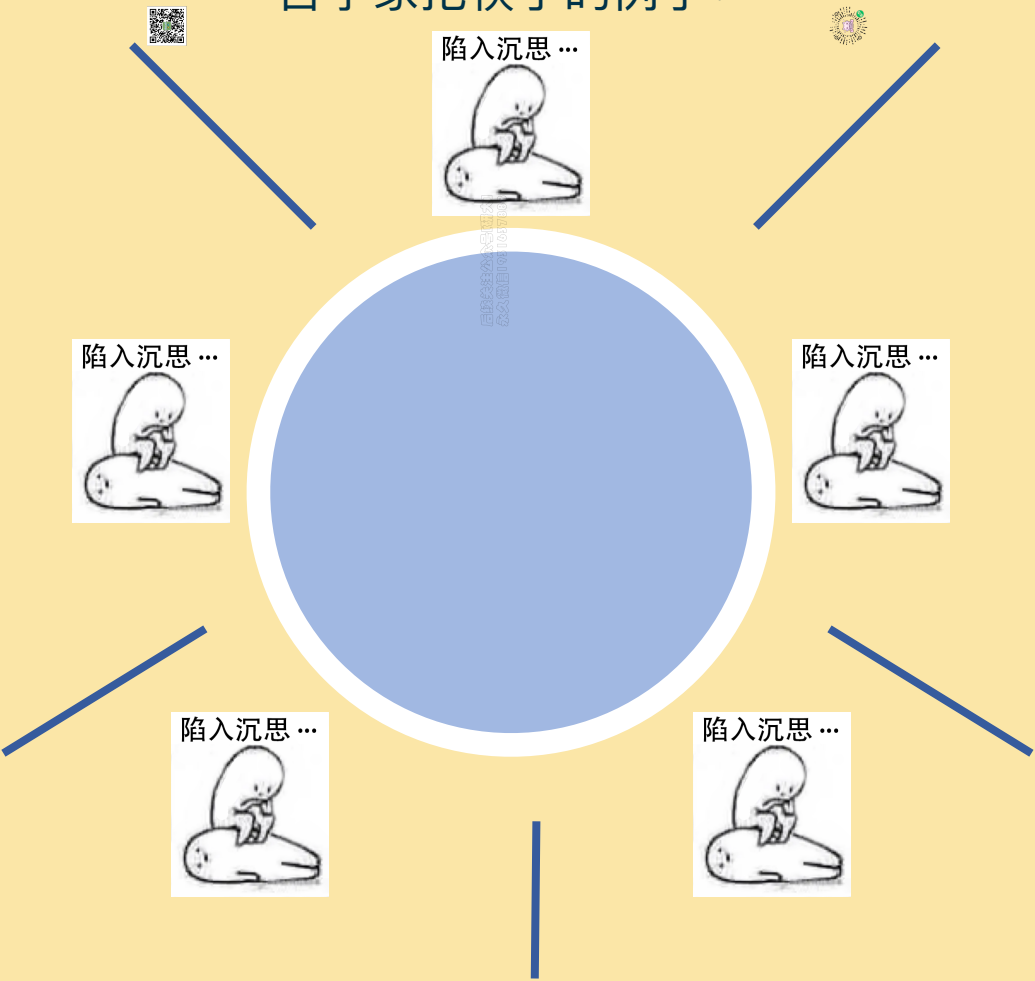


死锁

- 定义
- 必要条件
- 处理方法

如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

哲学家抢筷子的例子：





死锁

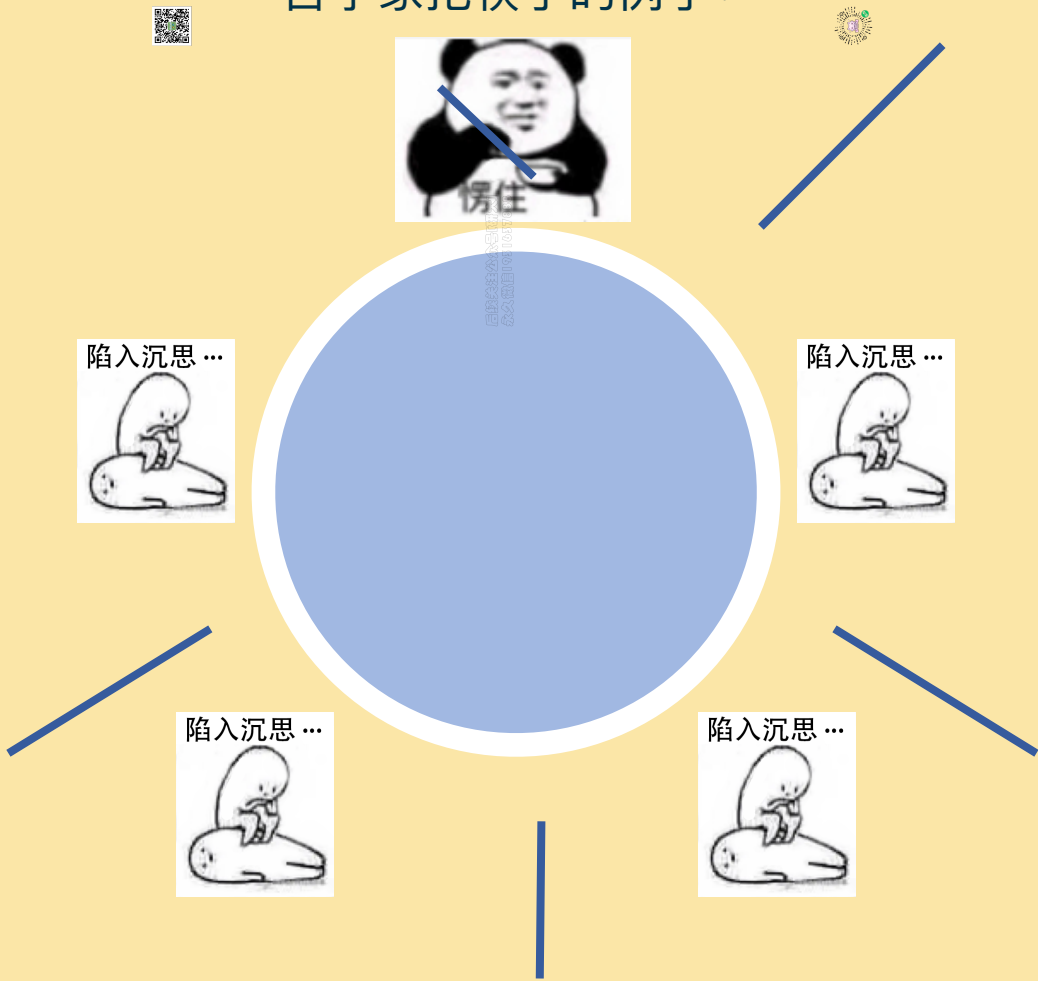
定义

必要条件

处理方法

如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

哲学家抢筷子的例子：





死锁

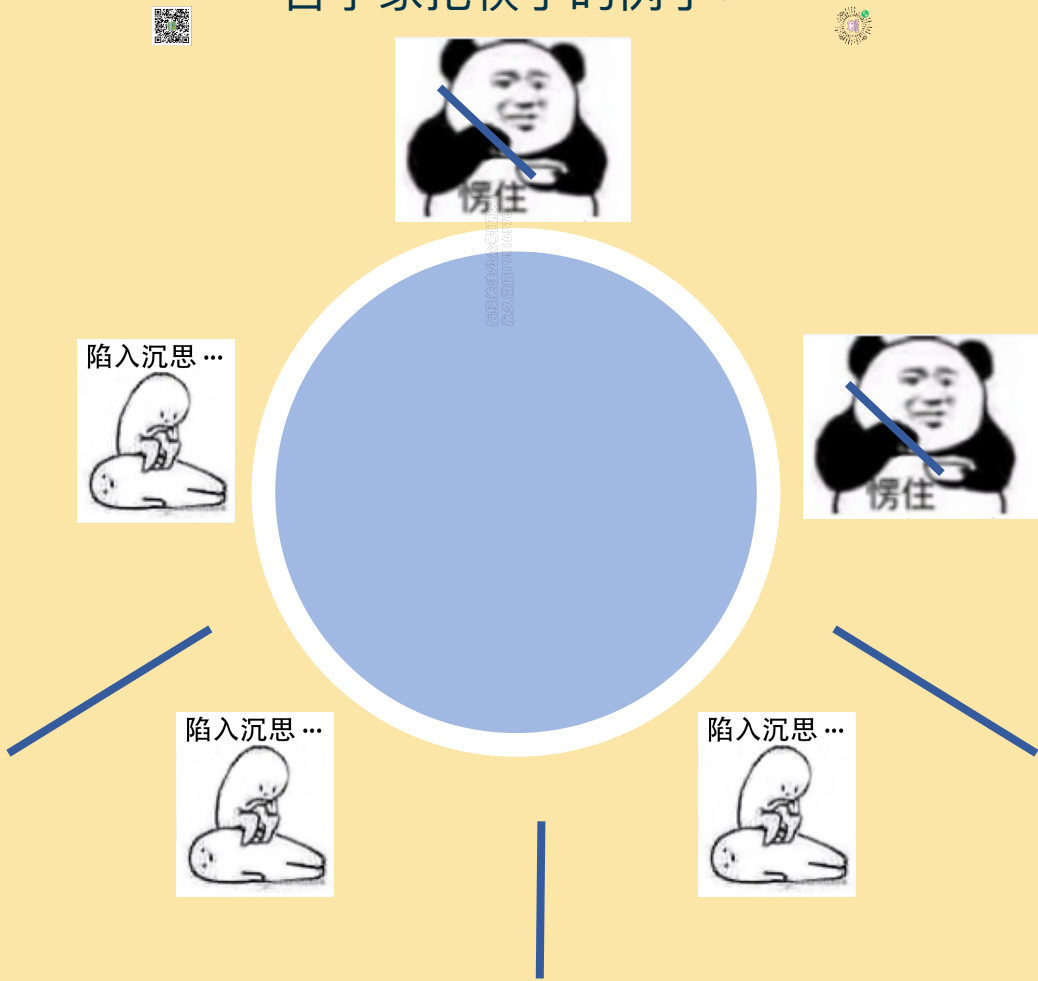
定义

必要条件

处理方法

如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

哲学家抢筷子的例子：





死锁

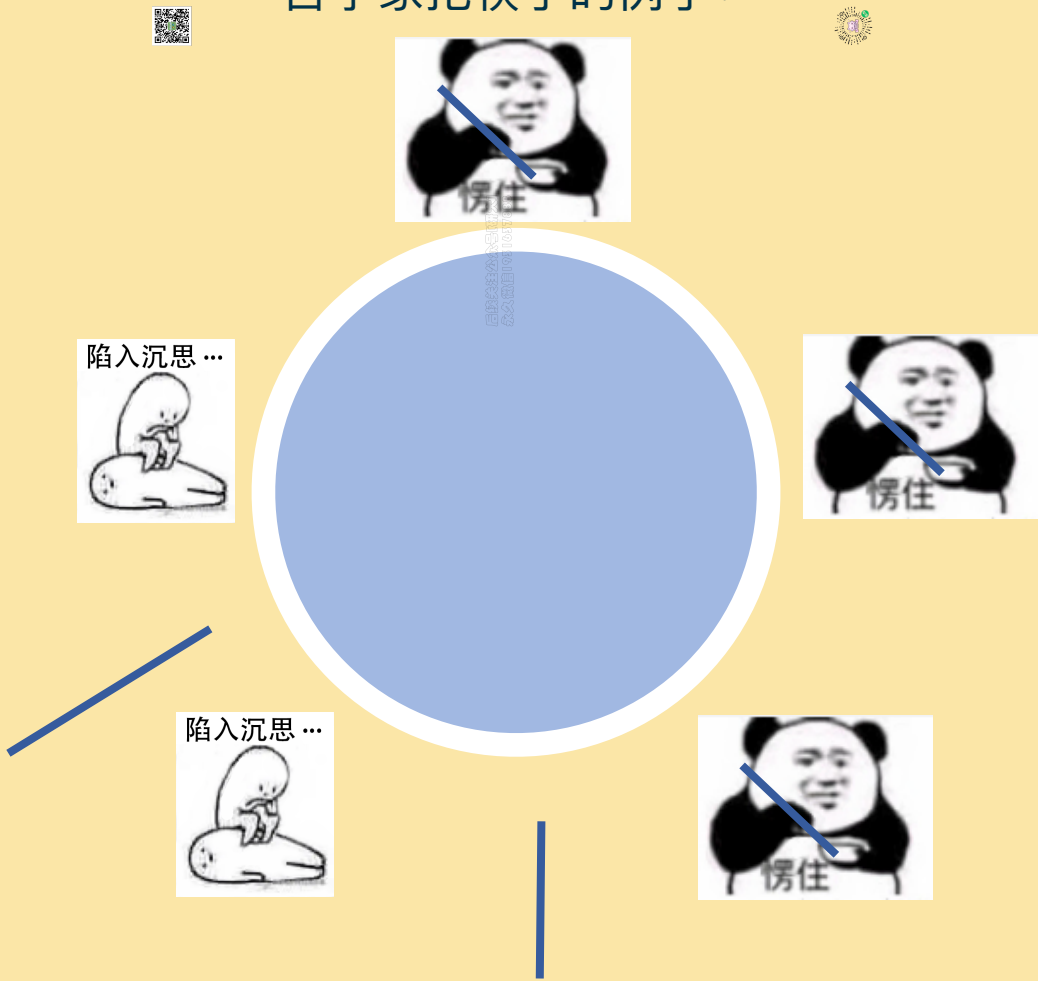
定义

必要条件

处理方法

如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

哲学家抢筷子的例子：





死锁

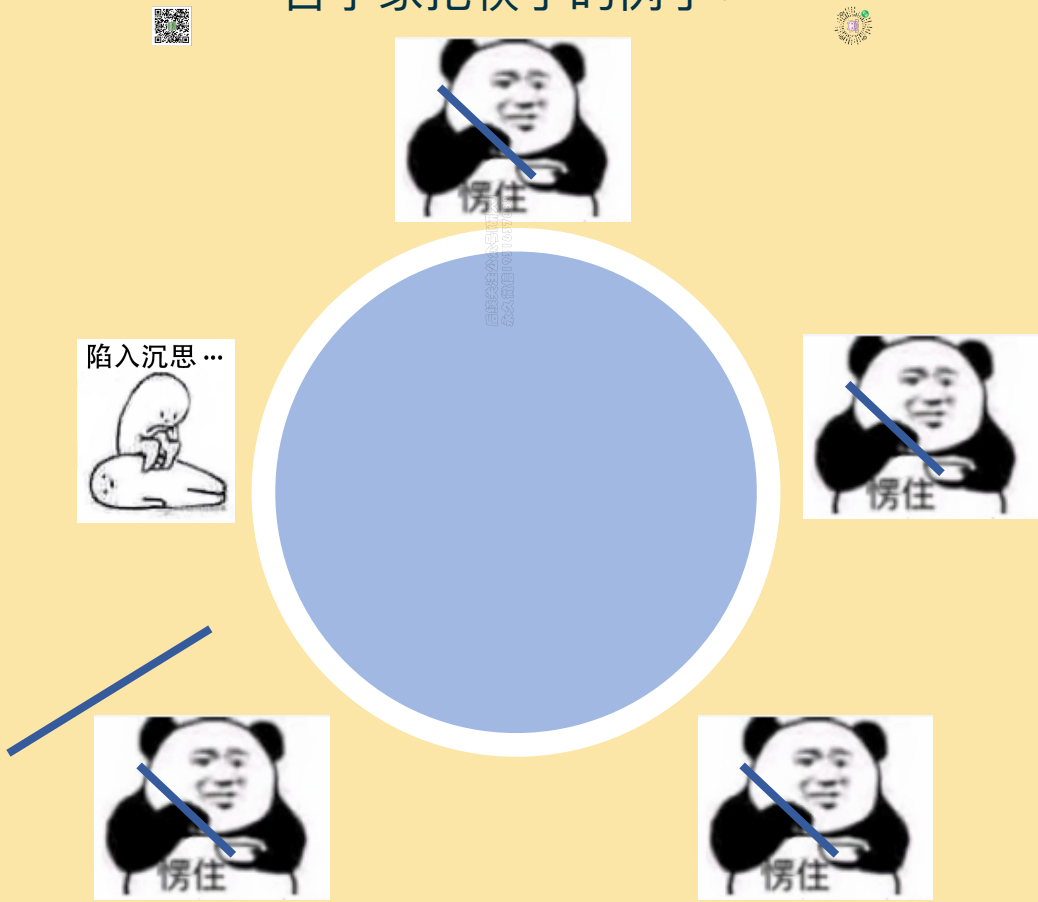
定义

必要条件

处理方法

如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

哲学家抢筷子的例子：





死锁

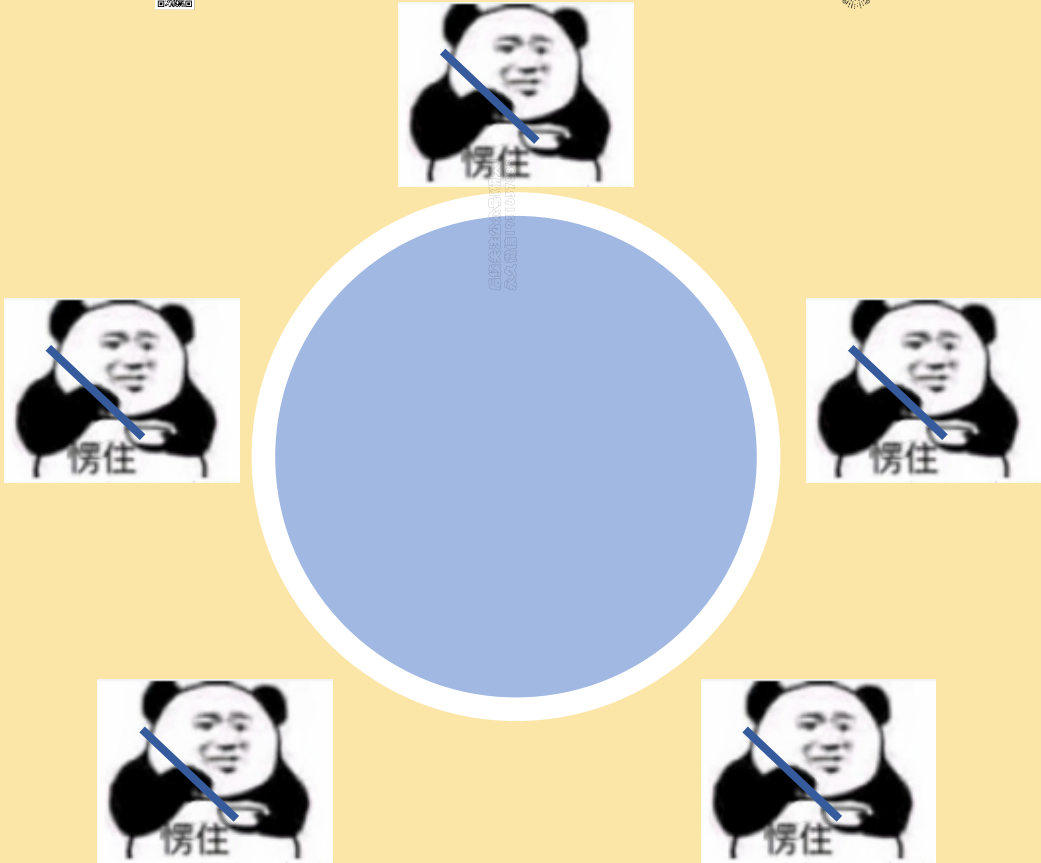
定义

必要条件

处理方法

如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

哲学家抢筷子的例子：





死锁

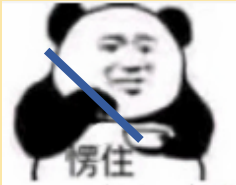
定义

必要条件

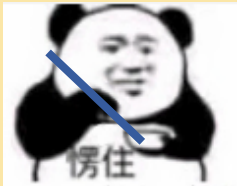
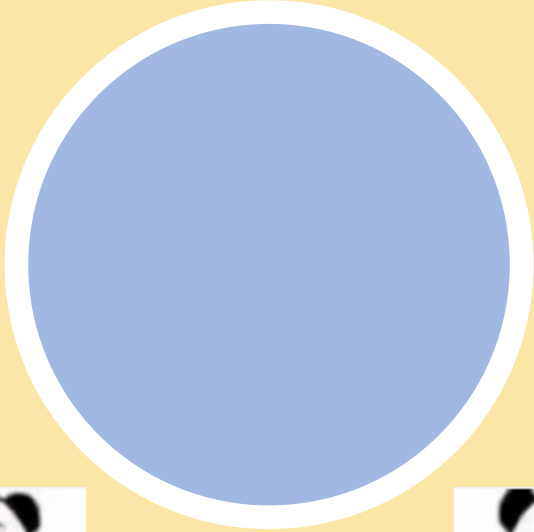
处理方法

如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

哲学家抢筷子的例子：



哲学家抢筷子的例子



每个哲学家都拿着自己左边的筷子
都在等待右边的筷子
所有哲学家都不会主动释放自己左边的筷子
形成了死锁，所有人都没饭吃



死锁

定义

必要条件

处理方法

如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的



饥饿

当等待时间
(等CPU、等IO资源...)
给进程的推进带来明显影响时
称发生了饥饿
饥饿不代表系统已经死锁
但至少有一进程执行被无限推迟

死锁产生的原因

死锁产生的原因

1. 竞争不可抢占资源：系统中的不可抢占资源数量
不足以满足多个进程运行需要
进程运行过程中就会因竞争资源而陷入僵局
(对可抢占资源的竞争不会引起死锁)
2. 进程推进顺序不当：哲学家进餐问题中
若事先规定好合法的吃饭顺序也不会引起死锁



死锁

定义

必要条件

处理方法

互斥条件

请求和保持条件

不可抢占条件

循环等待条件

死锁

- 定义
- 必要条件
- 处理方法
- 互斥条件
- 请求和保持条件
- 不可抢占条件
- 循环等待条件

一段时间内，进程分配到的资源只能被自己占用
如果有其他进程此时请求该资源，则请求进程只能等待



死锁

定义

必要条件

处理方法

互斥条件

请求和保持条件

不可抢占条件

循环等待条件

进程已经保持了至少一个资源，但又提出了新的资源请求
而该资源已被其他进程占有，此时请求进程被阻塞，但是对已有资源保持不放



死锁条件：请求与保持

进程已经保持了至少一个资源，但又提出了新的资源请求



而该资源已被其他进程占有



此时 P1 被阻塞



总结：进程在请求新资源时，因被占用而阻塞，但对已有资源保持不放。

死锁

定义

必要条件

处理方法

互斥条件

请求和保持条件

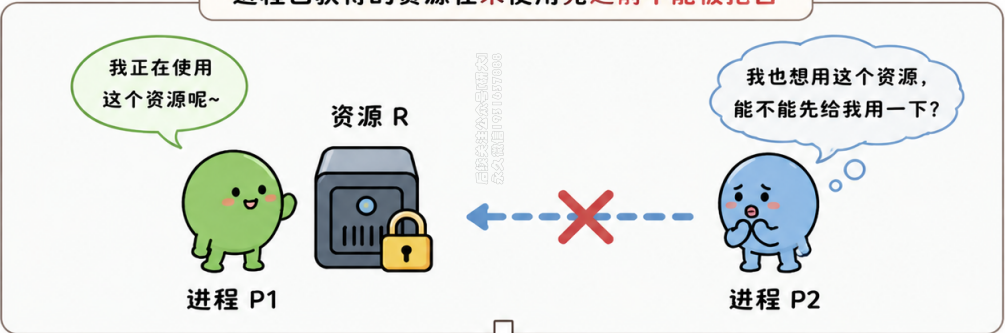
不可抢占条件

循环等待条件

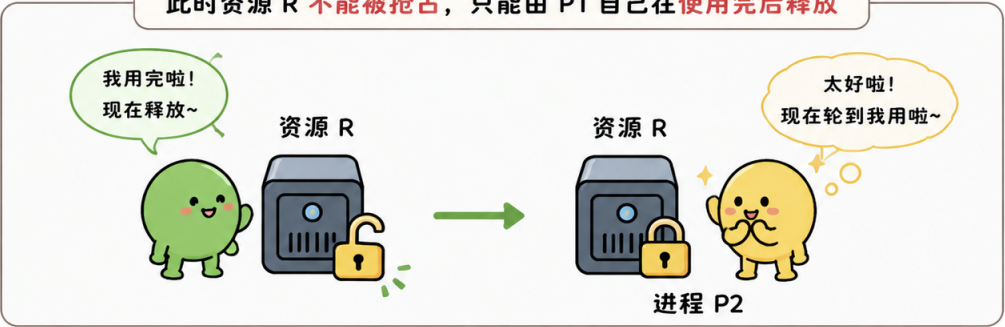
进程已获得的资源在未使用完之前不能被抢占
只能在进程使用完时由自己释放

★ 死锁条件：不可抢占

进程已获得的资源在**未使用完之前**不能被抢占



此时资源 R **不能被抢占**，只能由 P1 自己在**使用完后**释放



总结：资源在进程**未使用完之前**，不能被其他进程抢占，只能在进程**使用完**时由自己释放。

死锁

定义

必要条件

处理方法

互斥条件

请求和保持条件

不可抢占条件

循环等待条件

发生死锁时必然存在一个进程-资源的循环链
P1等P2资源， P2等P3.....





死锁

定义

必要条件

处理方法

互斥条件

请求和保持条件

不可抢占条件

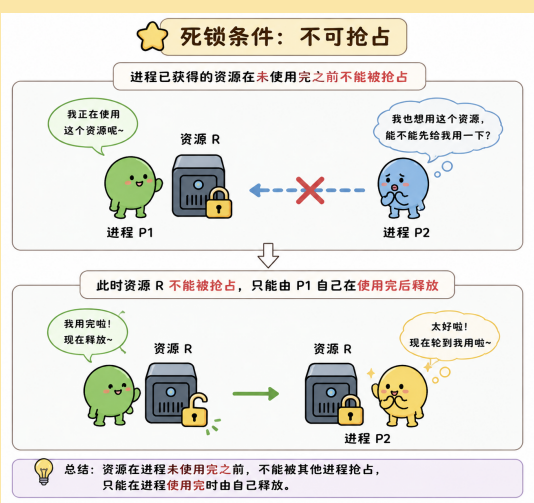
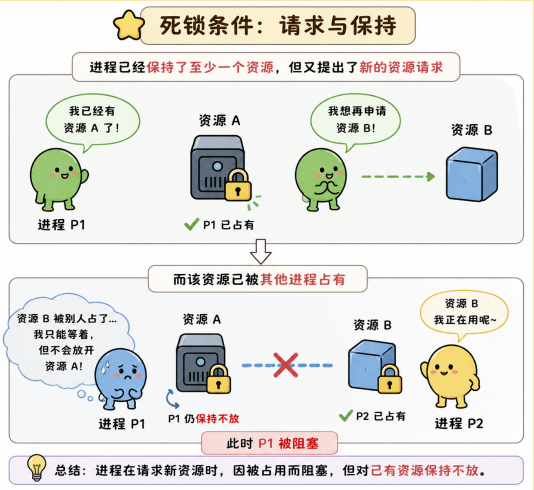
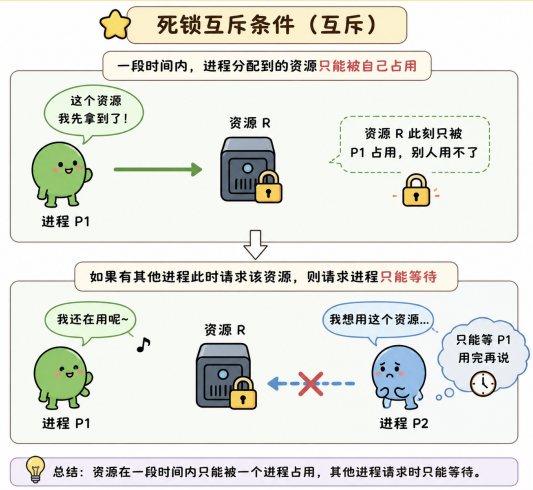
循环等待条件

一段时间内
进程分配到的资源只能被自己占用
如果有其他进程此时请求该资源
则请求进程只能等待

进程已经保持了至少一个资源
但又提出了新的资源请求
而该资源已被其他进程占有
此时请求进程被阻塞
但是对已有资源保持不放

进程已获得的资源在未使用完之前不能被抢占
只能在进程使用完时由自己释放

发生死锁时必然存在一个进程-资源的循环链
P1等P2资源, P2等P3.....





死锁

- 定义
- 必要条件
- 处理方法



预防死锁



避免死锁

检测死锁

解除死锁



死锁

- 定义
- 必要条件
- 处理方法

- 预防死锁
- 避免死锁
- 检测死锁
- 解除死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁
预防死锁是一种比较容易实现的办法，已被广泛使用



互斥条件不可被破坏，因为就是有临界资源只允许被互斥使用，我们还是来看看破坏别的吧



死锁

- 定义
- 必要条件
- 处理方法

- 预防死锁
- 避免死锁
- 检测死锁
- 解除死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁
预防死锁是一种比较容易实现的办法，已被广泛使用

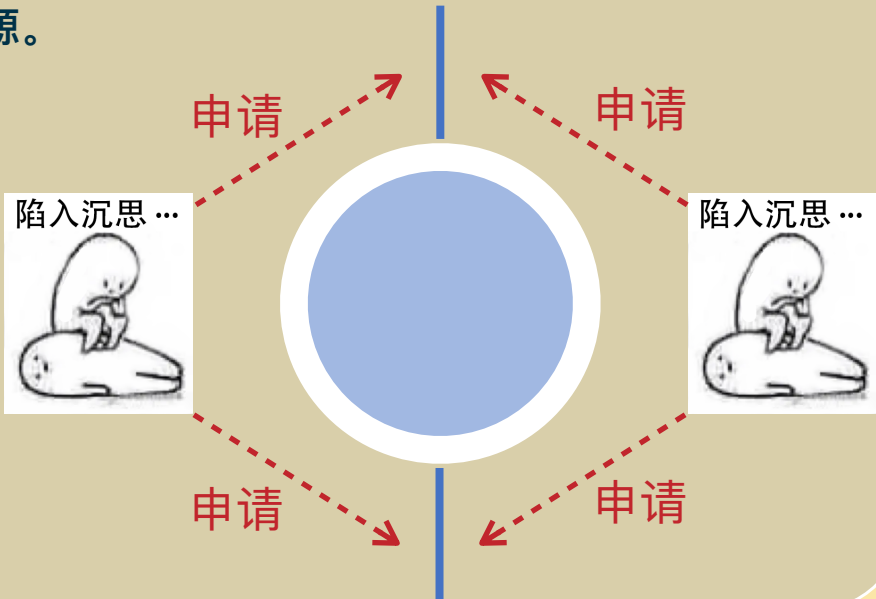
破坏请求和保持条件 方法一

请求和保持条件

可以让所有进程在开始运行之前
一次性地申请其在整个运行过程中需要的所有资源
若系统有足够的资源分给它，就给该进程所有它运行过程中需要的资源。

这样该进程在运行期间就不会在提出资源要求，破坏了“请求”条件

在申请资源时
只要有一种资源不能满足该进程要求
系统就**不会给该进程其他任何资源**
也破坏了“保持”条件，从而预防死锁发生





死锁

- 定义
- 必要条件
- 处理方法

- 预防死锁
- 避免死锁
- 检测死锁
- 解除死锁



通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁
预防死锁是一种比较容易实现的办法，已被广泛使用

破坏请求和保持条件 方法二

请求和保持条件

可以让进程申请，并分配给他初期运行时所需资源以方便进程运行
在进程释放出所有已分配资源以后才允许进程申请新的资源
这样能提高设备的利用率



死锁

- 定义
- 必要条件
- 处理方法

- 预防死锁
- 避免死锁
- 检测死锁
- 解除死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁
预防死锁是一种比较容易实现的办法，已被广泛使用



互斥条件不可被破坏，因为就是有临界资源只允许被互斥使用，我们还是来看看破坏别的吧



死锁

定义

必要条件

处理方法

预防死锁

避免死锁

检测死锁

解除死锁



通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁
预防死锁是一种比较容易实现的办法，已被广泛使用

破坏不可抢占条件

规定一个已经保持了某些不可被抢占资源的进程
提出新的资源请求不能被满足时
必须释放已经保持的所有资源
需要时再重新申请

该方法实现复杂，且需付出很大代价

不可抢占条件



死锁

定义

必要条件

处理方法

预防死锁

避免死锁

检测死锁

解除死锁



通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁
预防死锁是一种比较容易实现的办法，已被广泛使用

破坏循环等待条件

对系统的资源进行线性排序

规定每个进程**必须按照序号递增的顺序请求资源**

若拥有高序号资源以后又想要低序号资源，则需要释放掉所有高序号资源，再申请低序号资源

这样就破坏了“循环等待”的条件

循环等待条件

这种方式因为需要为资源编号，就限制了新类型设备的增加；作业使用顺序与系统预期不同也会导致资源浪费

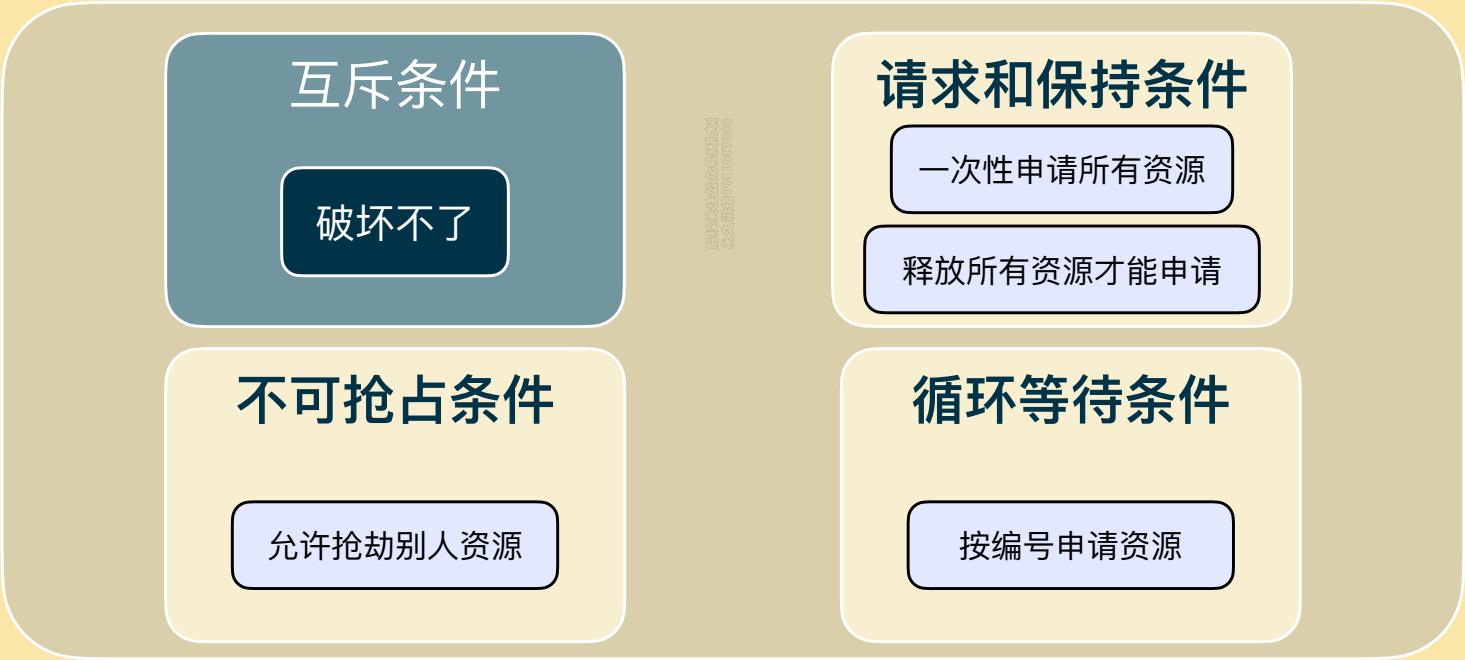


死锁

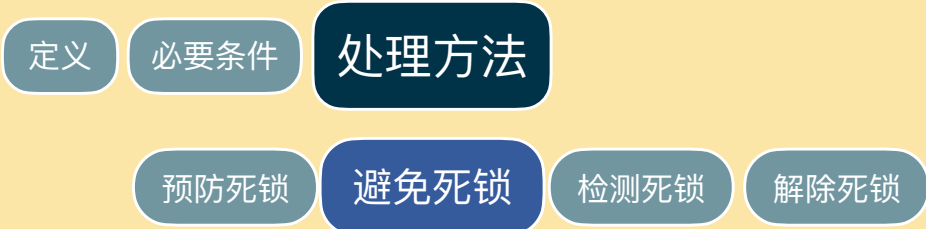
- 定义
- 必要条件
- 处理方法

- 预防死锁
- 避免死锁
- 检测死锁
- 解除死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁
预防死锁是一种比较容易实现的办法，已被广泛使用



死锁



银行家算法

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁
这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

安全状态：如果系统能按某种进程推进顺序为每个进程分配其所需资源
满足每个进程对资源的最大需求，使每个进程都可顺利地
完成这样的进程推进顺序($P_1, P_2 \dots P_n$)称为安全序列

不安全状态：如果找不到安全序列，那系统此时是**不安全状态**
进入**不安全状态**就可能发生死锁（也有可能不发生）
若系统处于**安全状态**，系统便不会进入死锁状态

某资源T0时刻资源分配情况如图，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	

定义

必要条件

处理方法

预防死锁

避免死锁

检测死锁

解除死锁

银行家算法

避免死锁=防止系统进入**不安全状态**

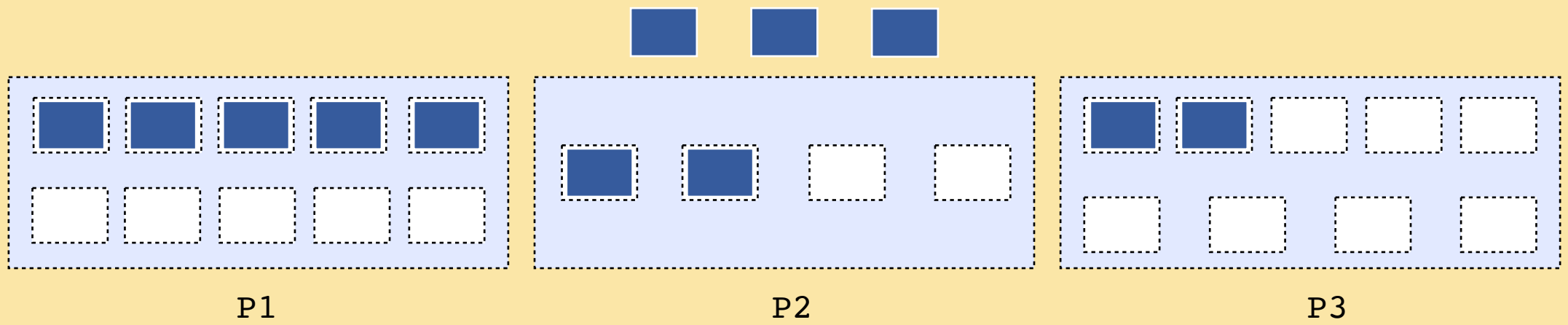
安全状态：合法分配资源的推进顺序 $(P_1, P_2 \dots P_n)$ 称为安全序列

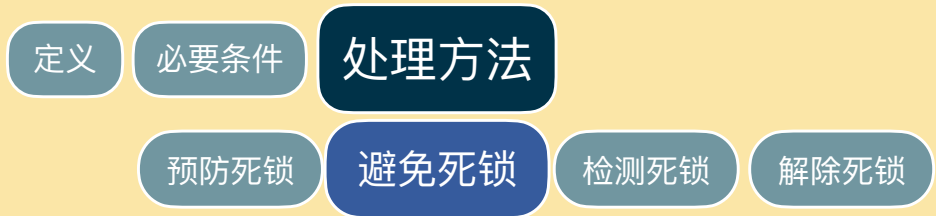
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**

某资源T0时刻资源分配情况如图，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	

进程	资源需求 Need
P ₁	5
P ₂	2
P ₃	7





银行家算法

避免死锁=防止系统进入**不安全状态**

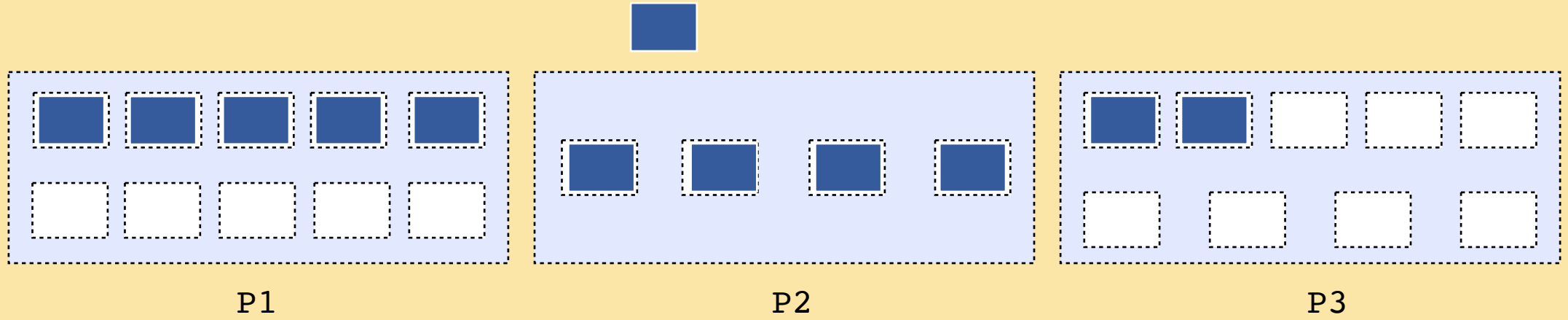
安全状态：合法分配资源的推进顺序 $(P_1, P_2 \dots P_n)$ 称为安全序列

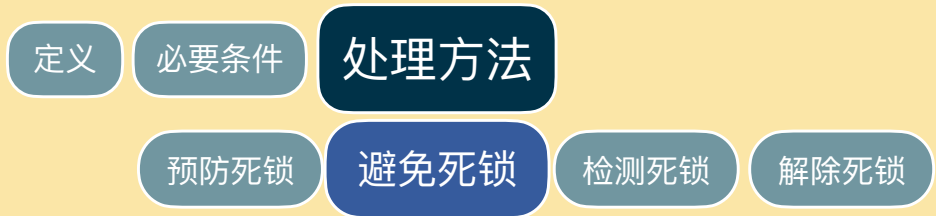
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**

某资源T0时刻资源分配情况如图，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	

进程	资源需求 Need
P ₁	5
P ₂	0
P ₃	7





银行家算法

避免死锁=防止系统进入**不安全状态**

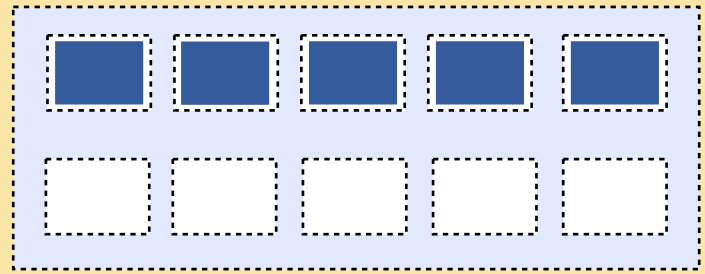
安全状态：合法分配资源的推进顺序 ($P_1, P_2 \dots P_n$) 称为安全序列

不安全状态：如果找不到安全序列，那系统此时是**不安全状态**

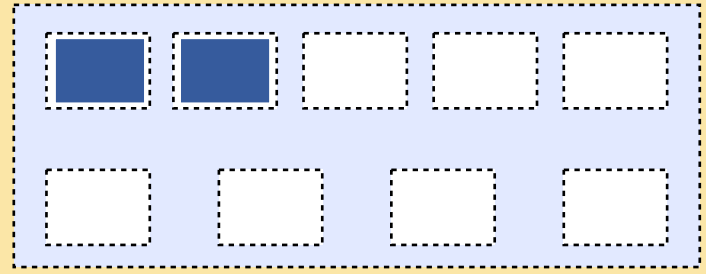
某资源T0时刻资源分配情况如图，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	

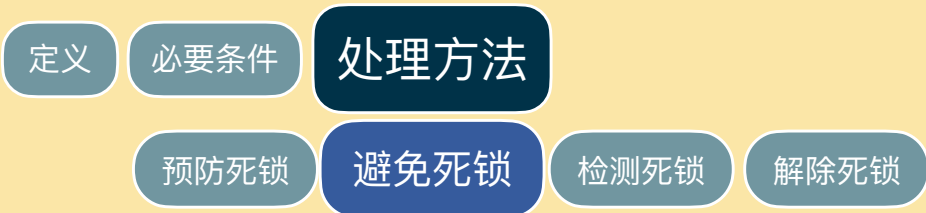
进程	资源需求 Need
P ₁	5
P ₂	0
P ₃	7



P1



P3



银行家算法

避免死锁=防止系统进入**不安全状态**

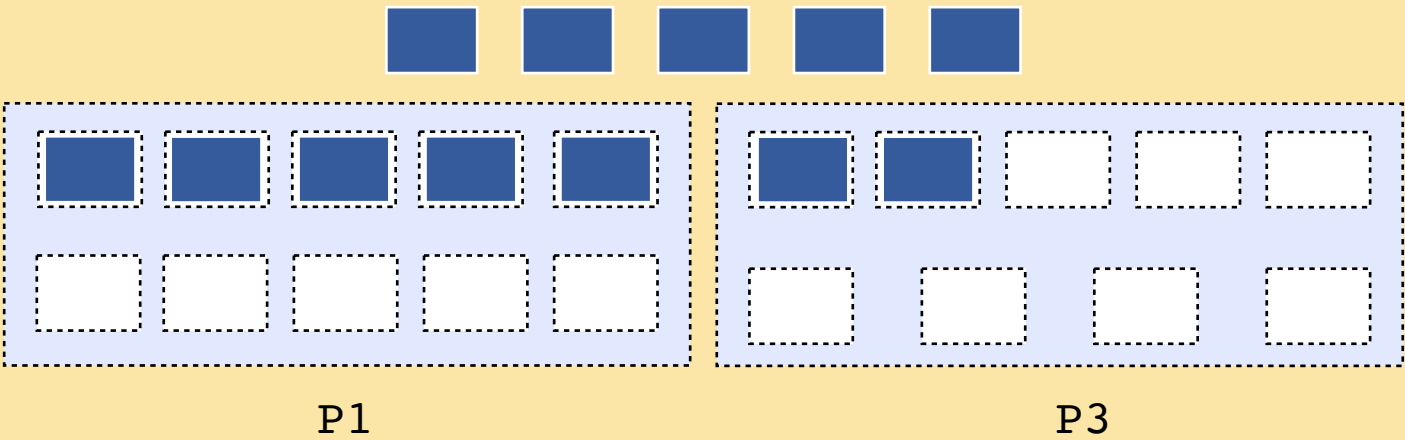
安全状态：合法分配资源的推进顺序 ($P_1, P_2 \dots P_n$) 称为安全序列

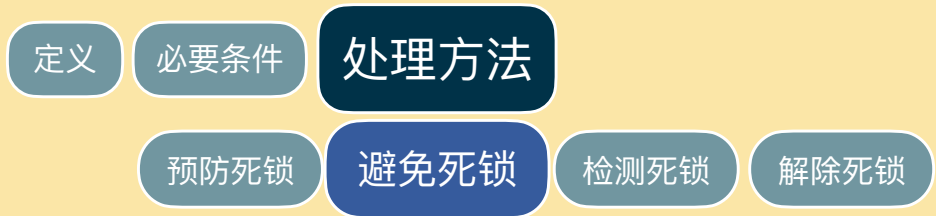
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**

某资源T0时刻资源分配情况如图，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	

进程	资源需求 Need
P ₁	5
P ₂	0
P ₃	7





银行家算法

避免死锁=防止系统进入**不安全状态**

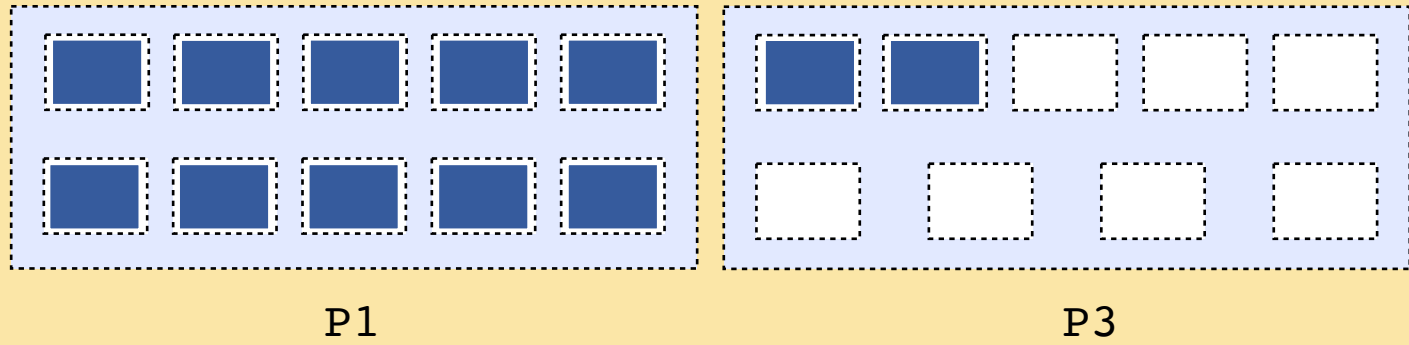
安全状态：合法分配资源的推进顺序 ($P_1, P_2 \dots P_n$) 称为安全序列

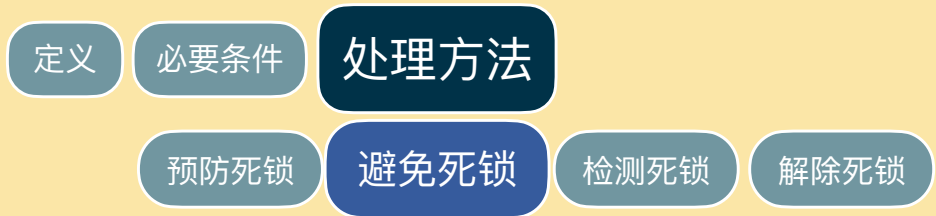
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**

某资源T0时刻资源分配情况如图，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	

进程	资源需求 Need
P ₁	0
P ₂	0
P ₃	7





银行家算法

避免死锁=防止系统进入**不安全状态**

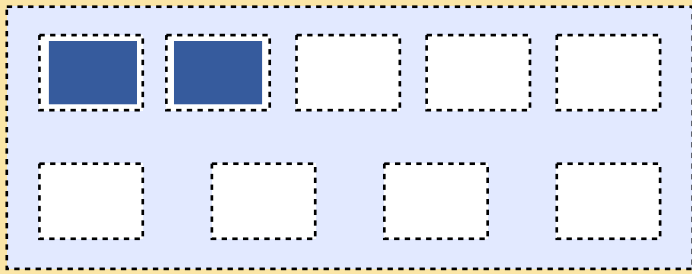
安全状态：合法分配资源的推进顺序 ($P_1, P_2 \dots P_n$) 称为安全序列

不安全状态：如果找不到安全序列，那系统此时是**不安全状态**

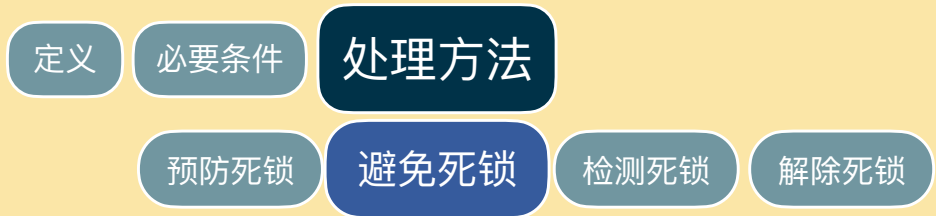
某资源T0时刻资源分配情况如图，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	

进程	资源需求 Need
P ₁	0
P ₂	0
P ₃	7



P3



银行家算法

避免死锁=防止系统进入**不安全状态**

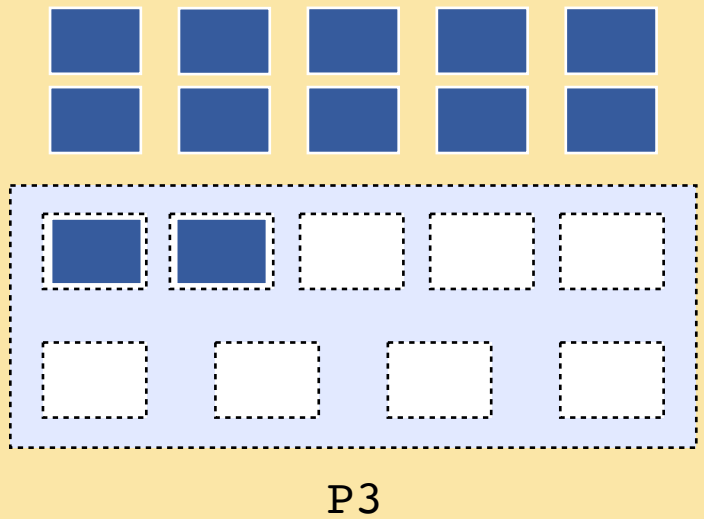
安全状态：合法分配资源的推进顺序 ($P_1, P_2 \dots P_n$) 称为安全序列

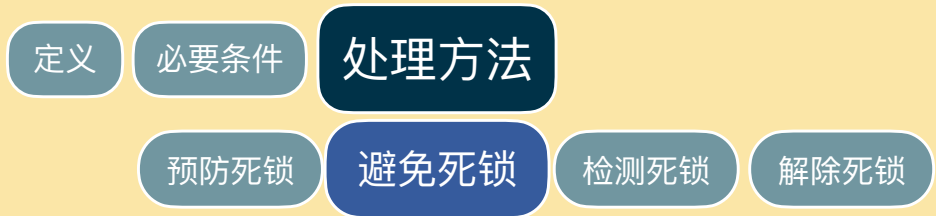
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**

某资源T0时刻资源分配情况如图，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P_1	10	5	3
P_2	4	2	
P_3	9	2	

进程	资源需求 Need
P_1	0
P_2	0
P_3	7





银行家算法

避免死锁=防止系统进入**不安全状态**

安全状态：合法分配资源的推进顺序 ($P_1, P_2 \dots P_n$) 称为安全序列

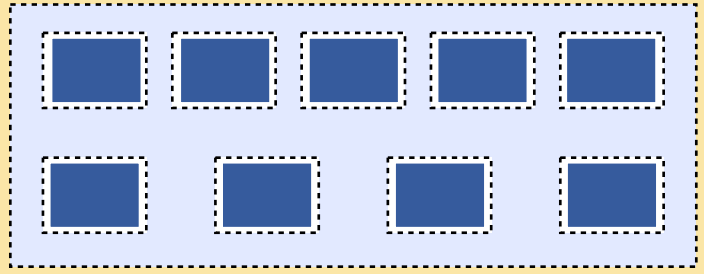
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**

某资源T0时刻资源分配情况如图，现在系统是安全状态吗？

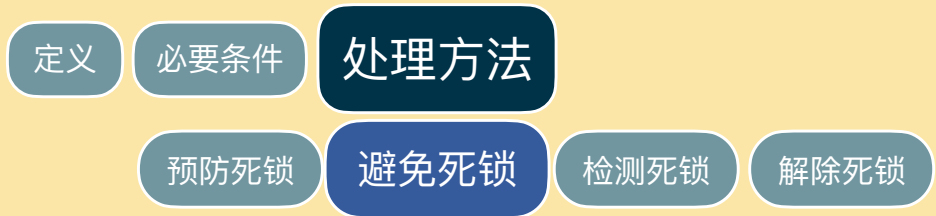
进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	



进程	资源需求 Need
P ₁	0
P ₂	0
P ₃	0



P3



银行家算法

避免死锁=防止系统进入**不安全状态**

安全状态：合法分配资源的推进顺序 ($P_1, P_2 \dots P_n$) 称为安全序列

不安全状态：如果找不到安全序列，那系统此时是**不安全状态**

某资源T0时刻资源分配情况如图，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P_1	10	5	3
P_2	4	2	
P_3	9	2	

存在安全序列 $P_2 \rightarrow P_1 \rightarrow P_3$
系统处于安全状态

如果不按照安全序列执行的话会让系统进入不安全状态
不安全状态可能发⽣死锁



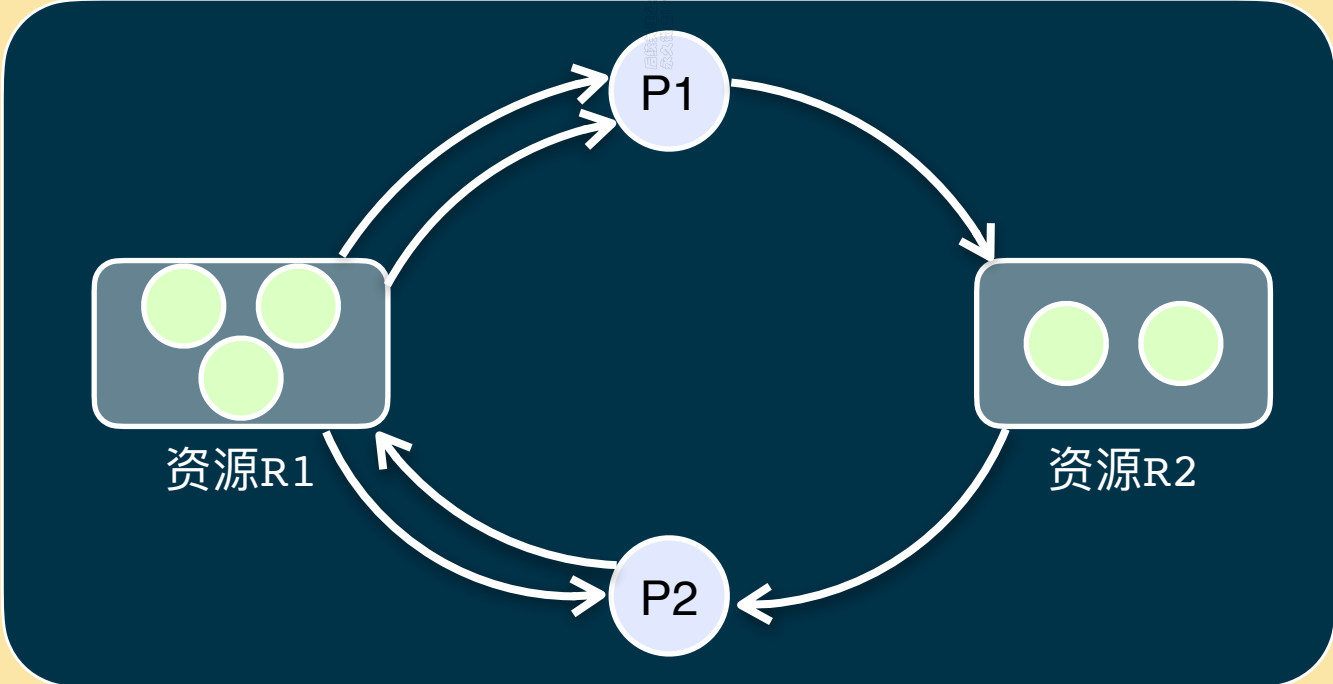
死锁

- 定义
- 必要条件
- 处理方法

- 预防死锁
- 避免死锁
- 检测死锁
- 解除死锁

资源分配图

由一组结点 N 和一组边 E 组成的一个对偶 $G=(N, E)$
把 N 分为两个互斥子集，一组进程节点 P 和一组资源节点 R
每一个边都连着 P 和 R 中的一个结点， P 指向 R 的是资源请求边
 R 指向 P 的是资源分配边





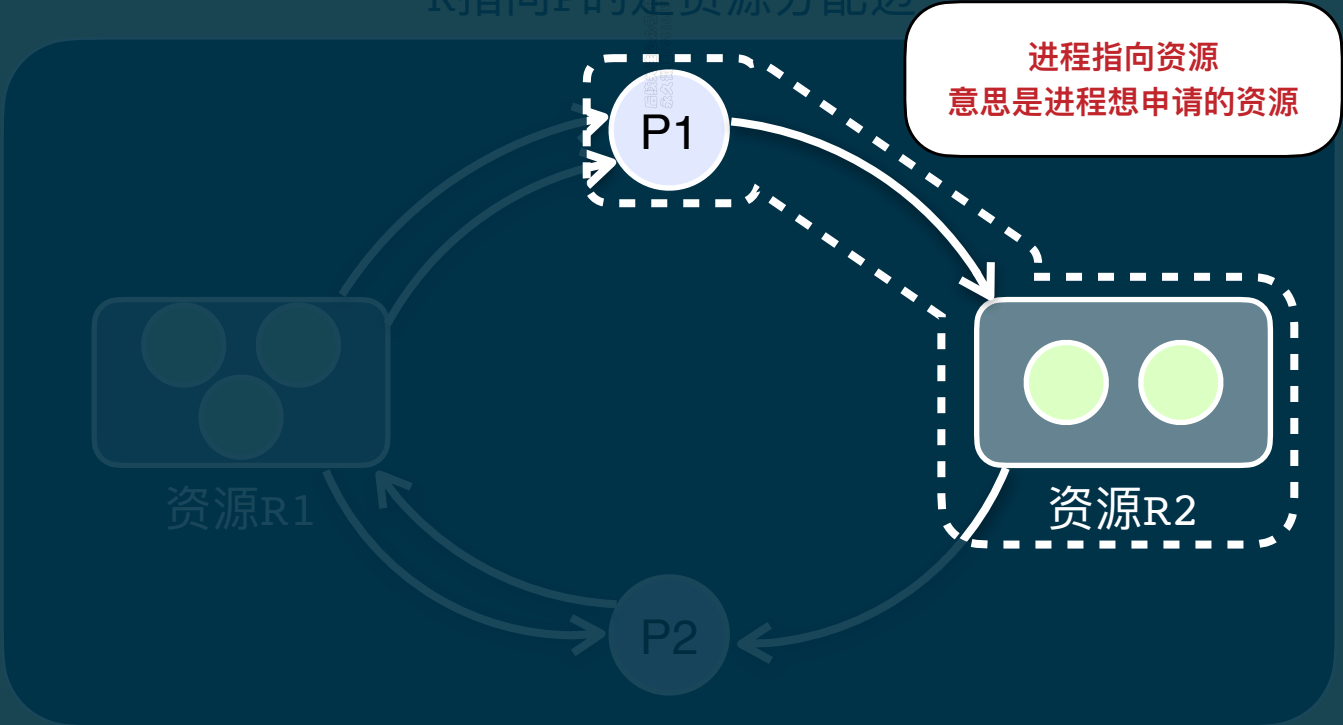
死锁

- 定义
- 必要条件
- 处理方法

- 预防死锁
- 避免死锁
- 检测死锁
- 解除死锁

资源分配图

由一组结点 N 和一组边 E 组成的一个对偶 $G=(N,E)$
把 N 分为两个互斥子集，一组进程节点 P 和一组资源节点 R
每一个边都连着 P 和 R 中的一个结点， P 指向 R 的是资源请求边
 R 指向 P 的是资源分配边





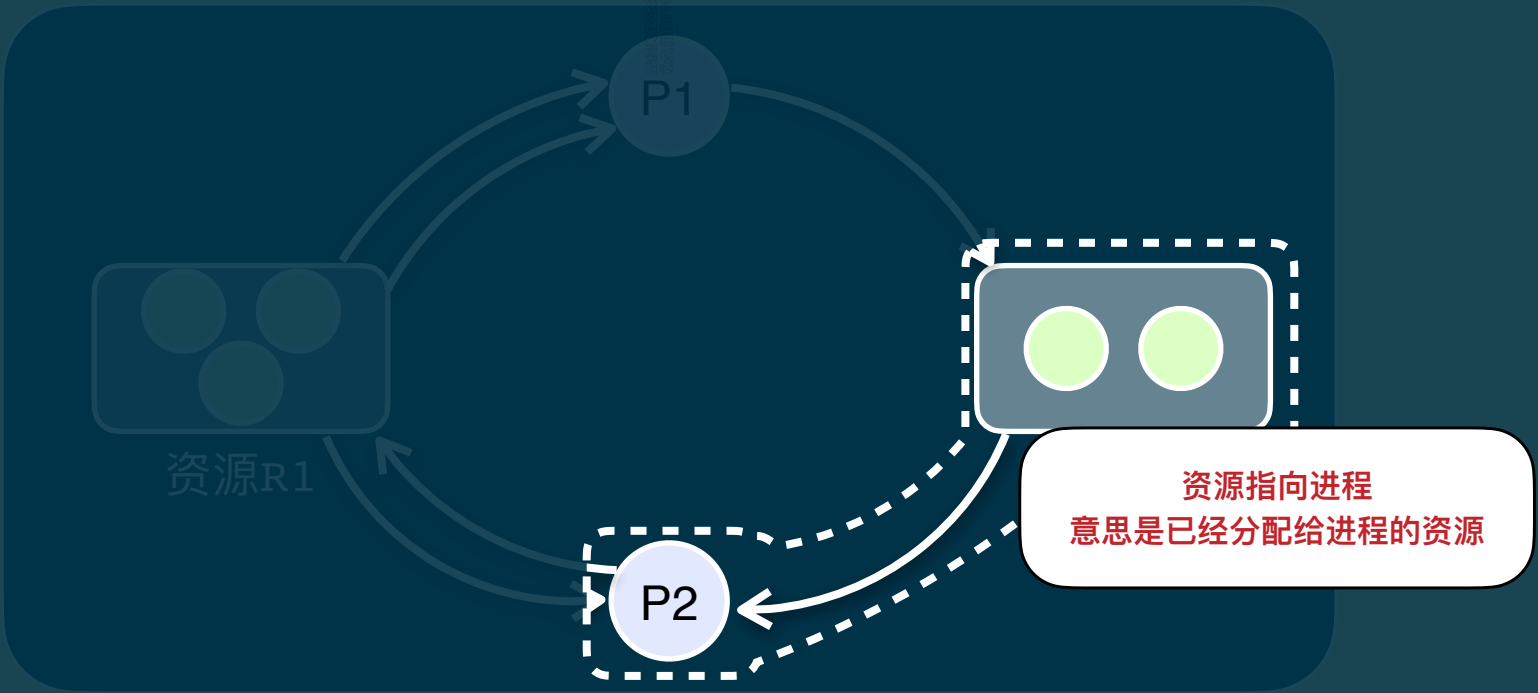
死锁

定义 必要条件 处理方法

预防死锁 避免死锁 检测死锁 解除死锁

资源分配图

由一组结点 N 和一组边 E 组成的一个对偶 $G=(N,E)$
把 N 分为两个互斥子集，一组进程节点 P 和一组资源节点 R
每一个边都连着 P 和 R 中的一个结点， P 指向 R 的是资源请求边
 R 指向 P 的是资源分配边





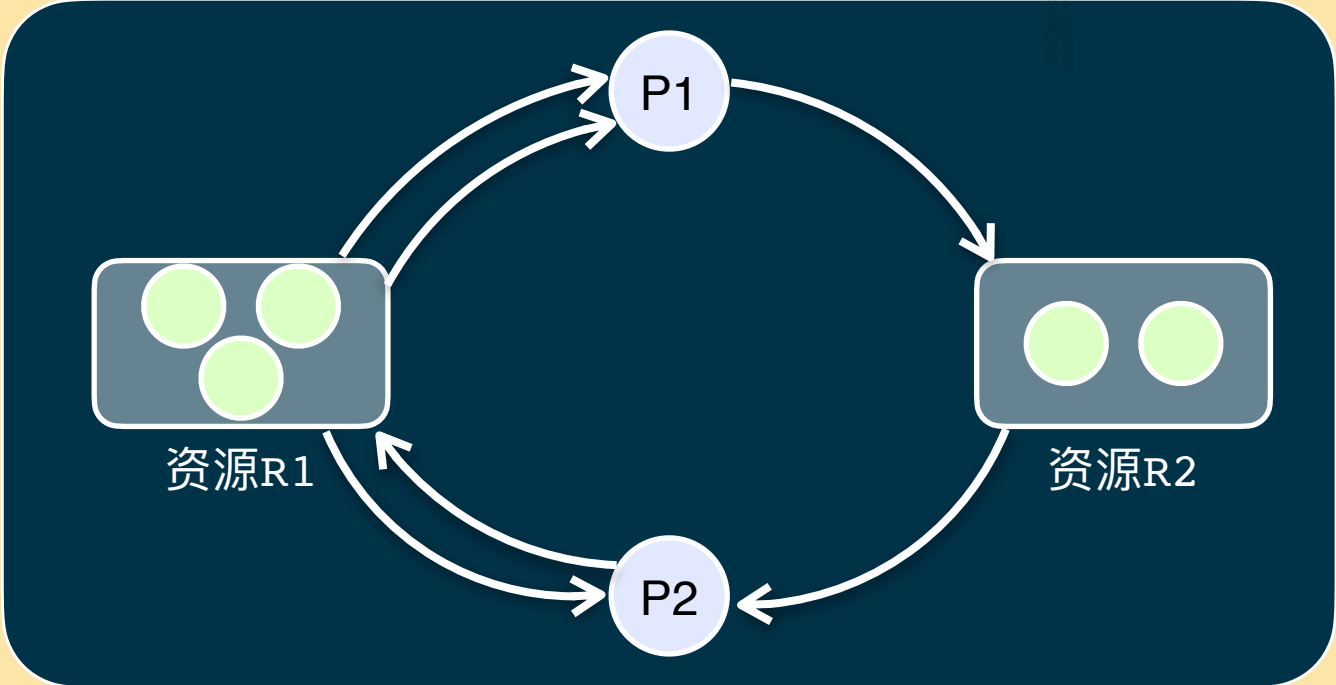
死锁

定义 必要条件 **处理方法**

预防死锁 避免死锁 **检测死锁** 解除死锁

资源分配图

由一组结点 N 和一组边 E 组成的一个对偶 $G=(N, E)$
把 N 分为两个互斥子集，一组进程节点 P 和一组资源节点 R
每一个边都连着 P 和 R 中的一个结点， P 指向 R 的是资源请求边
 R 指向 P 的是资源分配边



R1分了两个给P1
R2分了一个给P2

P1申请R2
P2申请R1



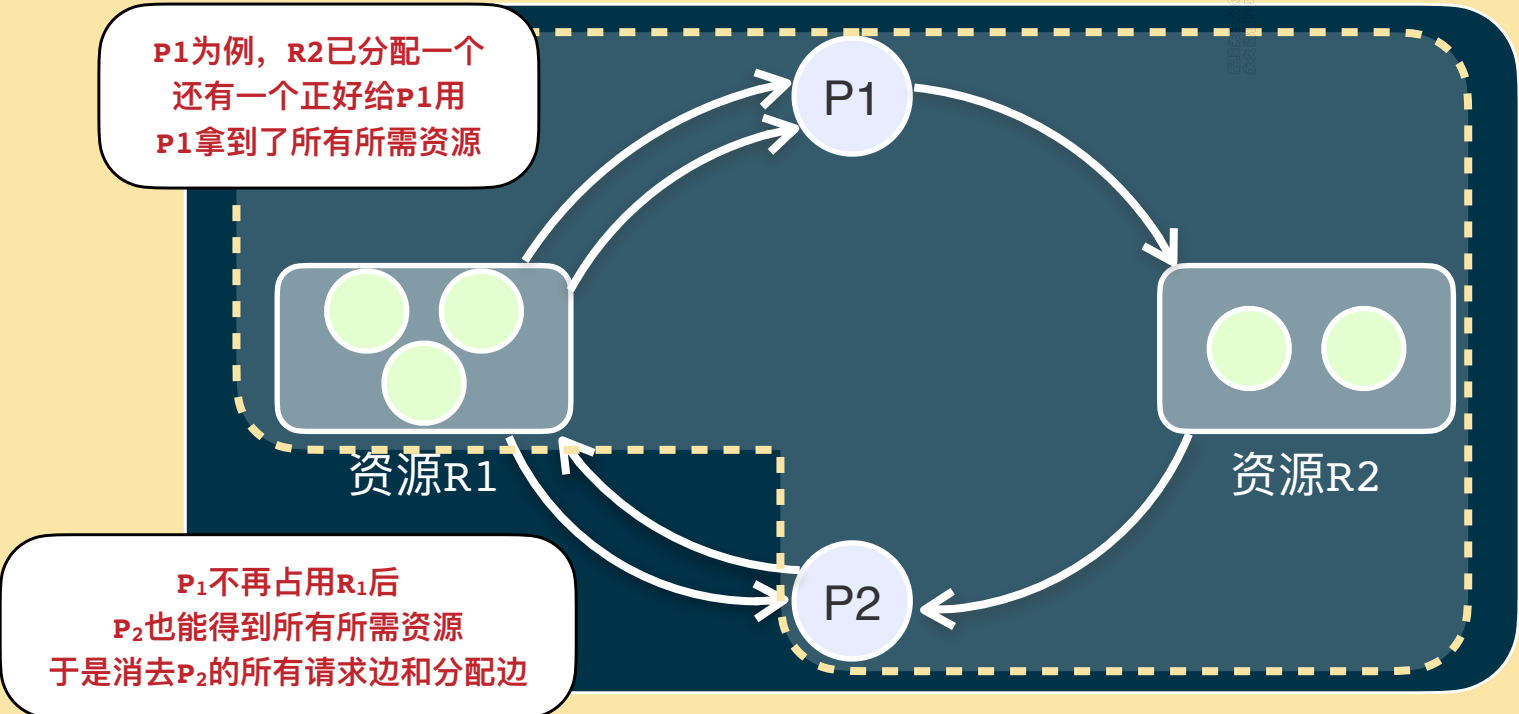
死锁

定义 必要条件 **处理方法**

预防死锁 避免死锁 **检测死锁** 解除死锁

资源分配图的化简

- 1. 找出一个不阻塞也不独立的进程结点 P_i ，检查一切顺利的话能否给该进程所有需要的资源让其运行完毕
 可以就释放其占用的所有资源，也就是消去他的所有请求边和分配边
- 2. P_1 释放资源后就可以使 P_2 获得资源继续运行，把图中所有进程依次按照这种方法化简



**R1分了两个给P1
R2分了一个给P2**

**P1申请R2
P2申请R1**



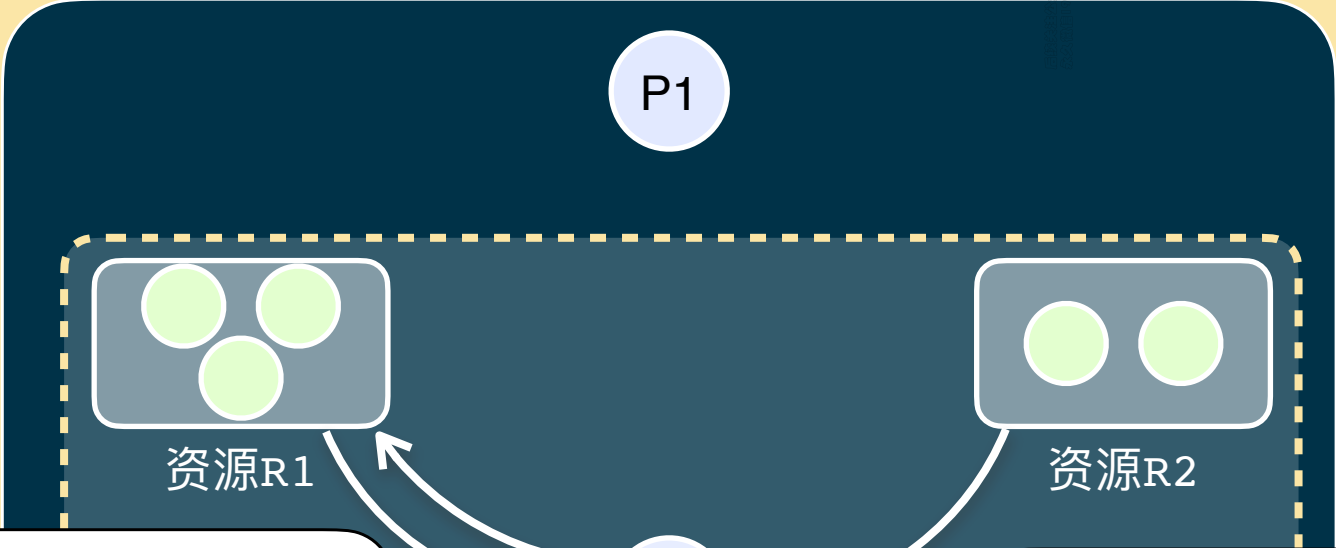
死锁

定义 必要条件 **处理方法**

预防死锁 避免死锁 **检测死锁** 解除死锁

资源分配图的化简

- 1. 找出一个不阻塞也不独立的进程结点 P_i ，检查一切顺利的话能否给该进程所有需要的资源让其运行完毕
 可以就释放其占用的所有资源，也就是消去他的所有请求边和分配边
- 2. P_1 释放资源后就可以使 P_2 获得资源继续运行，把图中所有进程依次按照这种方法化简



R1分了两个给P1
R2分了一个给P2

P1申请R2
P2申请R1

P1不再占用R1后
P2也能得到所有所需资源
于是消去P2的所有请求边和分配边

能把所有边消除，即为可简化的
消除不了所有边，则是死锁状态



死锁

定义

必要条件

处理方法

预防死锁

避免死锁

检测死锁

解除死锁

1. 资源剥夺法：挂起一些死锁进程，抢占他的资源，分配给其他死锁进程

2. 撤销进程法：强制撤销部分或者全部死锁进程并且剥夺资源
可以按照优先级和撤销代价来考虑撤销顺序，实现简单但是代价较大

3. 进程回退法：让一个或多个死锁进程回退到足以回避死锁的地步，进程回退时自愿释放资源而不是被剥夺
但要求系统保持进程历史信息，设置还原点

了解即可



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

【2009统考真题】某计算机系统中有8台打印机，由 κ 个进程竞争使用，每个进程最多需要3台打印机，该系统可能发生死锁的 κ 的最小值是

A. 2 B. 3 C. 4 D. 5



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

【2009统考真题】某计算机系统中有8台打印机，由 κ 个进程竞争使用，每个进程最多需要3台打印机
该系统可能发生死锁的 κ 的最小值是

A. 2

B. 3

C. 4

D. 5

$$R \geq n(m - 1) + 1$$

资源量
Resource

进程数量
 n

每个进程要求的资源数量
 m



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

【2009统考真题】某计算机系统中有8台打印机，由 κ 个进程竞争使用，每个进程最多需要3台打印机
该系统可能发生死锁的 κ 的最小值是

A. 2

B. 3

C. 4

D. 5

$$R \geq n(m - 1) + 1$$

资源量
Resource

进程数量
n

每个进程要求的资源数量
m

资源总量 $\geq$ 每个进程都差一个就能满足的数量 + 1
就不会发生死锁